Synchronization Tools

Introduction

- **The Problem**: In a multithreaded or multiprocess system, concurrent access to shared data can lead to data inconsistency
- **The Goal (Synchronization):** To provide mechanisms that ensure the orderly execution of cooperating processes so that data consistency is maintained
- **Key Focus**: This section covers the critical section problem, software/hardware solutions, and advanced synchronization primitives like semaphores and monitors

Background and the Critical-Section Problem

- **Race Condition:** A situation where several processes access and manipulate the same data concurrently, and the outcome of the execution depends on the order in which the access takes place
  - Example: A bank debit/credit operation where the final balance is incorrect due to interleaved updates.
- **Critical Section:** The segment of code where a process or thread accesses and modifies shared data
- **The Critical-Section Problem:** Designing a protocol that processes can use to cooperate.
  - This protocol must guarantee that when one process is executing in its critical section, no other process is allowed to execute in its critical section.

The Critical-Section Problem (Solution Requirements)

- Any solution to the Critical-Section Problem must satisfy three requirements:
  - **Mutual Exclusion:** If process is executing in its critical section, then no other processes can be executing in their critical sections
  - **Progress:** If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in deciding which will enter next
    - This selection cannot be postponed indefinitely
  - **Bounded Waiting:** There must be a limit on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Peterson's Solution

- **Description:** A classic software-based solution to the critical-section problem for two processes (and)
- **Mechanism:** Uses two shared variables:
  - turn: An integer variable indicating whose turn it is to enter the critical section
  - flag[2]: A Boolean array where flag[i] = true means is ready to enter the critical section
- **The Protocol (for Process):**
  - See program at the side
- **Proof:** Peterson's solution satisfies all three requirements:
  - **Mutual Exclusion, Progress, and Bounded Waiting**
- **Limitation:** It is limited to only **two processes** and does not work well on modern computer architectures where memory access is optimized and reordered

```
do {
    flag[i] = true;   // Indicate
I want to enter
    turn = j;         // Give
permission to the other process
    while (flag[j] && turn == j);
// Wait if other process wants in
AND it's their turn
    // CRITICAL SECTION
    flag[i] = false;  // Exit
    // REMAINDER SECTION
} while (true);
```

Synchronization Hardware

- **Hardware Solutions:** Modern computer architectures provide special **atomic** (uninterruptible) hardware instructions to simplify the critical-section problem and increase efficiency.
- **Atomic Operation:** An operation that is executed as a single, uninterruptible unit
- **TestAndSet() Instruction:** Atomically tests the value of a memory word and sets it to true (or 1)
  - See program at top side
- **CompareAndSwap() Instruction (CAS):** Atomically compares the value of a memory word with an expected value, and if they match, swaps the memory word with a new value
  - See program bottom side
- **Mutex Locks using Hardware:** These atomic instructions can be used to implement simple locks (called **mutex locks** in operating systems) to ensure mutual exclusion

```
boolean TestAndSet (boolean *target)
{
    boolean rv = *target;
    *target = true;
    return rv;
}
```

Test and set instruction

```
int CompareAndSwap (int *value, int
expected, int new_value) {
    int temp = *value;
    if (*value == expected)
        *value = new_value;
    return temp;
}
```

Compare and swap instruction

Mutex Locks

- **Mutex (Mutual Exclusion) Lock:** The simplest and most common synchronization tool
- A process must **acquire()** the lock before entering a critical section and **release()** the lock upon exiting
- **State:** A mutex lock has a Boolean variable, available, indicating if the lock is held (false) or not (true).
- **Acquire and Release Operations:**
- **Busy Waiting (Spinlock):** The process wishing to acquire the lock continuously loops until the lock is available
  - This wastes CPU cycles if the critical section is long
  - A lock that uses busy waiting is called a **spinlock**.
- **Benefit of Spinlocks:** Spinlocks are useful in **multicore systems** because no context switch is required when a thread waits for a lock, which is advantageous if the waiting time is expected to be very short.

Semaphores

- **Semaphore:** A synchronization primitive that provides a more sophisticated tool than simple mutex locks. It is an integer variable that is accessed only through two atomic operations: **`wait()`** and **`signal()`**
- **Operations:**
  - **`wait()`** **(or *P*):** Decrements the semaphore value. If the value becomes negative, the process is blocked
  - **`signal()`** **(or *V*):** Increments the semaphore value. If the value is less than or equal to zero, a waiting (blocked) process is unblocked
- **Types of Semaphores:**
  - **Counting Semaphore:** The integer value can range over an unrestricted domain. Used to control access to a resource pool with multiple instances
  - **Binary Semaphore:** The integer value can only be 0 or 1. Functions identically to a mutex lock
  - **Solving Busy Waiting:** Semaphores solve the busy-waiting problem by associating a **waiting queue** with the semaphore. When a process executes a `wait()` operation and finds the semaphore value , it is placed into this queue and its state is changed to blocked

Classic Synchronization Problems

- These problems are used as benchmarks to test the efficiency and correctness of new synchronization tools
  - Bounded-Buffer Problem
  - Readers-Writers Problem

The Bounded-Buffer Problem (Producer-Consumer)

- **Goal:** Synchronize a **Producer** (who produces items) and a **Consumer** (who consumes items) using a finite-sized buffer.
- **Semaphores Used:**
  - mutex: Binary semaphore for mutual exclusion (initial value = 1).
  - empty: Counting semaphore for the number of empty slots (initial value = Buffer size N).
  - full: Counting semaphore for the number of occupied slots (initial value = 0).

The Readers-Writers Problem

- **Goal:** Allow multiple processes to concurrently read shared data, but only one process can write to the data at a time
- **Priority:** Two variations exist: prioritizing readers (no reader waits for a writer to finish) or prioritizing writers (no new readers can start if a writer is waiting)

The Dining-Philosophers Problem

- **Goal:** Illustrate the problem of deadlock. philosophers sitting at a round table, each needing two chopsticks (resources) to eat, but only chopsticks available.
- **Deadlock Condition:** If all philosophers pick up one chopstick simultaneously, they will all wait indefinitely for the second, leading to **deadlock**.

Monitors

- **Monitor:** A high-level synchronization construct that is designed to simplify concurrent programming and avoid common errors associated with semaphores.
- **Encapsulation:** A monitor is an abstract data type that contains:
  - A set of programmer-defined operations (methods)
  - An encapsulated set of shared variables
- **Synchronization mechanism** (condition variables) built into the structure
- **Mutual Exclusion Guarantee:** The monitor construct guarantees that **only one process can be active within the monitor at any given time**. This built-in mutual exclusion greatly simplifies programming
- **Condition Variables:** Used within a monitor to allow a process to wait for a specific condition to become true. The two operations are:
  - wait(): Suspends the calling process until another process invokes signal()
  - signal(): Resumes one suspended process. If no process is waiting, the operation has no effect

Deadlocks and Starvation

- **Deadlock:** A condition where two or more processes are waiting indefinitely for an event that can only be caused by one of the waiting processes
  - Example: Two processes, and , each hold a resource and wait for the other process to release the resource.
- **Starvation (Indefinite Blocking):** A process may wait indefinitely within a semaphore's waiting queue, even though other processes are repeatedly entering and leaving the critical section
  - Unlike deadlock, the system is still making progress, just not for the starved process
  - Solution: Ensure that a process waiting in the queue for a resource is eventually served

Synchronization Examples

- **Pthreads Synchronization:** The **Pthreads API** provides a variety of synchronization tools, including:
  - pthread_mutex_t: Mutex locks for mutual exclusion
  - pthread_cond_t: Condition variables for complex waiting
  - Semaphores are also available in Pthreads
- **Windows Synchronization:** Windows provides several mechanisms, including:
  - **Dispatcher Objects:** General synchronization objects (mutants, semaphores, events, timers)
  - **Critical-Section Objects:** A simpler, user-mode version of the mutex lock for use only within a single process
- **Linux Synchronization:** Linux provides a variety of tools, including:
  - **Spinlocks:** Used when the waiting time is expected to be short
  - **Mutexes:** Used when the waiting time may be longer
  - **Semaphores:** The full semaphore mechanism is available
  - **Reader-Writer Locks:** Optimized for the readers-writers problem